

1. Навигация и жизненный цикл форм (критично)

1.1. Постоянное создание новых экземпляров вместо возврата

При закрытии почти любой дочерней формы создаётся **новый** экземпляр главной формы (`new MainAdminForm()` , `new MainUserForm()` , `new MainWorkerForm()`), а старая скрывается (`Hide()`).

Последствия:

- Потеря состояния (фильтры, выбранные строки, прокрутка, открытые вкладки).
- Утечки памяти (старые формы не всегда корректно уничтожаются).
- Повторная загрузка всех данных из БД при каждом возврате.
- Сложный и хрупкий код с `FormClosed` -обработчиками, которые закрывают предыдущую форму.

1.2. Флаг `DataRequests.IsOpen` как «модальность»

Используется глобальный флаг, чтобы запретить открытие второй копии некоторых окон (`ForgotPassword`, `About`, `DataTicket` и т.д.).

Проблемы:

- Легко «залипает» в `true` при аварийном закрытии или исключении.
- Не блокирует форму по-настоящему (можно кликать под ней).
- Не решает проблему нескольких экземпляров других форм.

1.3. `TelegramForm` всегда создаётся и показывается

В `LoginForm` создаётся `TelegramForm` и вызывается `Show()` . Форма прозрачная (красная + `TransparencyKey`), но всё равно существует как окно.

UX-эффект: лишний процесс, возможное мерцание, странное поведение `Alt+Tab`.

2. Авторизация и «Запомнить меня»

2.1. Фильтрация ввода в реальном времени

C#

```
TextBoxPassword.Text = Regex.Replace(TextBoxPassword.Text, "[^A-Za-z0-9]",  
"");  
TextBoxLogin.Text = Regex.Replace(...);
```

При каждом нажатии клавиши текст перезаписывается. Курсор прыгает в конец, невозможно нормально редактировать середину строки. Спецсимволы (даже допустимые в паролях) мгновенно удаляются.

2.2. «Запомнить меня» — логика и UX

- Сохранение происходит только при установке галочки (`CheckedChanged`), а не при успешном входе.
- Если пользователь ввёл логин/пароль, поставил галочку, а потом изменил пароль — в файл записывается старое значение.
- Нет очистки файла при снятии галочки.
- Нет проверки, что файл существует и корректно разбит по : (при повреждении — падение).
- Файл лежит в `%TEMP%\SupportDesk\loginPassword.txt` в открытом виде.

2.3. Счётчик попыток входа (`attemptLogin`)

- Счётчик живёт только в экземпляре `LoginForm` . После закрытия/пересоздания формы сбрасывается.
- Блокировка происходит только если логин **существует** в БД. При несуществующем логине попытки не считаются.
- После 3 неудачных попыток аккаунт блокируется, но пользователь не видит, сколько попыток осталось.

2.4. Проверка «слабого» пароля после входа

`CheckValidatePassword()` вызывается **после успешного входа**. Пользователь уже попал в систему, но получает сообщение «Измените пароль...». Нет принудительного перехода на смену пароля.

3. Создание заявки (`CreateNewTicketForm`)

3.1. Значение «Не назначен» для исполнителя

```
C#
```

```
comboBoxExecutor.SelectedItem = "Не назначен";
```

Это значение **никогда не добавляется** в `Items`. Поэтому `SelectedItem` всегда остаётся `null` (или предыдущим значением). В БД и Telegram уходит пустая строка или мусор.

3.2. Валидация времени

- Проверяются только границы 8–18 и минуты 0–59.
- Нет проверки, что время начала < времени окончания (кроме полного совпадения).
- Можно указать интервал 17:50 – 08:10 (через полночь) — формально проходит.
- Поля времени — обычные `TextBox` с фильтром цифр, а не `DateTimePicker` / `MaskedTextBox`. Легко ввести одну цифру.

3.3. Обязательные поля

Кнопка активируется при заполнении всех текстовых полей, но **не проверяется**, выбран ли элемент в `ComboBox` (можно оставить пустой текст).

3.4. Возврат после создания

Снова создаётся новый экземпляр главной формы вместо возврата к существующему.

4. Регистрация пользователя

4.1. Генератор пароля

```
C#
```

```
Random rnd = new Random();
```

Классический `Random` без `seed` — предсказуемый. Длина фиксирована (8 символов). Нет специальных символов, хотя политика «слабых» паролей их не требует.

4.2. Роль всегда USER

При регистрации из формы жёстко ставится `groupUser = "USER"`. Админ не может сразу создать WORKER/ADMIN через эту форму (нужно потом менять вручную).

4.3. Показ пароля в MessageBox

После регистрации пароль показывается в обычном MessageBox. Любой, кто смотрит на экран, его видит.

4.4. Нет проверки уникальности логина до вставки

Проверка уникальности, судя по коду, отсутствует или выполняется только на уровне исключения БД.

5. Восстановление пароля (ForgotPasswordForm)

- SQL-инъекция (конкатенация).
- Отправляется **текущий** пароль открытым текстом на почту.
- Нет rate-limit / защиты от перебора email+login.
- Сообщение об ошибке одинаковое при неверном логине и неверной почте (хорошая практика), но при успехе пароль уходит в открытом виде.

6. Общие UX-проблемы интерфейса

Проблема	Описание
Отсутствие стандартных кнопок окна	Все формы без системной рамки, перетаскивание через <code>WM_NCLBUTTONDOWN</code> . Нет кнопки «Свернуть» на многих формах, только кастомный крестик.
Нет подтверждения деструктивных действий	Удаление записей, блокировка пользователя часто без «Вы уверены?».
Дублирование кода перемещения окна	Один и тот же блок <code>FormControl</code> + <code>DllImport</code> скопирован в десятки форм.
MessageBoxUI	Кастомные сообщения, но часто с техническими текстами (<code>ex.Source</code> , <code>ex.Message</code>) — пользователь видит «System.Data.SQLite».
Нет индикации загрузки	При открытии главных форм идёт несколько <code>async</code> -запросов к БД, но нет <code>ProgressBar</code> / <code>Disabled</code> -состояния.

ComboBox без «пустого» значения	Многие списки не имеют пункта «— выберите —», поэтому первое значение выбирается автоматически и может быть отправлено по ошибке.
Жёсткие лимиты без предупреждения	<code>limitUsers = 500</code> , <code>limitTech = 5000</code> — пользователь узнаёт об ошибке только при попытке добавить.
Фильтрация ввода в <code>TextChanged</code>	Повсеместно. Ломает UX редактирования (курсор, вставка, Undo).
Нет горячих клавиш	Enter не всегда отправляет форму, Esc не закрывает.
Аватары и файлы	Пути к картинкам жёстко прописаны. При отсутствии файла показывается «Error» или пустота без понятного сообщения.

7. Логические ошибки данных и состояния

1. **Глобальные статические классы** (`DataUsers` , `DataRequests` , `DataSpecificUser`) используются как «сессия». При переключении между пользователями или открытии нескольких окон данные могут перезаписаться.
2. **Дата заявки** хранится как строка `"dd.MM.yyyy HH:mm"` + отдельно время. Сортировка и фильтрация по дате работают плохо.
3. **Статус «Новая»** и исполнитель «Не назначен» — магические строки, разбросанные по коду. Легко опечататься.
4. **Блокировка аккаунта** (`typeAccess = "LOCKED"`) — нет UI для разблокировки в очевидном месте (только через админку / прямое изменение БД).
5. **Повторный вход** через `EnterLoginPerUser` (автологин) пропускает проверку блокировки и слабого пароля в некоторых ветках.

8. Приоритеты исправления (UI/UX + логика)

Критично (ломает работу / вводит в заблуждение)

1. Добавить значение «Не назначен» в ComboBox исполнителей (или ставить `SelectedIndex = -1` + текст-подсказку).
2. Исправить навигацию: не создавать новые главные формы, а показывать уже существующую (`Show()` + `BringToFront()`).
3. Убрать перезапись `.Text` в `TextChanged` (использовать `KeyPress` / `Validating` или `MaskedTextBox`).

4. Сделать корректное сохранение «Запомнить меня» только после успешного входа и очистку при снятии галочки.

Высокий приоритет

1. Показывать оставшееся количество попыток входа.
2. После обнаружения слабого пароля — принудительно открывать форму смены пароля.
3. Добавить подтверждения на удаление и блокировку.
4. Валидация времени заявки: начало < конец, разумный интервал.

Средний приоритет

1. Единый базовый класс для форм с перетаскиванием и закрытием.
 2. Индикаторы загрузки.
 3. Нормальные DateTimePicker / TimePicker вместо четырёх TextBox.
 4. Убрать технические тексты исключений из MessageBox для пользователя.
-

Итог: приложение функционально, но UX сильно страдает из-за неправильного управления жизненным циклом форм, агрессивной фильтрации ввода, магических строк в ComboBox и отсутствия обратной связи. Большинство проблем легко исправляются без переписывания архитектуры, но требуют системного подхода (базовый Form-класс. единый сервис навигации. нормальная валидация).